



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey.

Maestría en Inteligencia Artificial Aplicada.

Pruebas de software y aseguramiento de la calidad

Actividad 5.2

Ejercicio de programación 2 y análisis estático.

Alumna: Gianmel Joannelly Hernández Tosta

Matrícula: A01795919

15/02/2026

Índice

Índice.....	1
Introducción.....	2
Ejercicio de programación 2.....	3
Solución.....	4
Bibliografía.....	12

Introducción

El aseguramiento de la calidad del software es un componente fundamental en el desarrollo de sistemas confiables, mantenibles y escalables. Dentro de este contexto, las pruebas estáticas y dinámicas permiten detectar defectos en etapas tempranas del ciclo de vida del software, reduciendo riesgos y mejorando la robustez del código. El uso de herramientas de análisis estático, junto con pruebas automatizadas, constituye una práctica esencial para garantizar el cumplimiento de estándares de calidad y buenas prácticas de programación.

La presente actividad tiene como objetivo desarrollar un programa en Python que calcule el total de ventas de una empresa a partir de dos archivos en formato JSON: un catálogo de precios y un registro de ventas. Además de implementar la lógica funcional solicitada, el ejercicio integra mecanismos de validación de datos, manejo de errores, medición del tiempo de ejecución y generación de resultados en archivo externo, cumpliendo con los requisitos establecidos en la especificación.

Adicionalmente, se aplicaron herramientas de análisis estático como **flake8** y **pylint** para asegurar el cumplimiento del estándar de codificación PEP8 y evaluar la calidad estructural del código. Asimismo, se implementaron pruebas unitarias utilizando **pytest**, permitiendo validar el correcto funcionamiento del cálculo de ventas. De esta manera, la actividad no solo aborda la solución funcional del problema planteado, sino que también integra prácticas formales de aseguramiento de la calidad del software, alineadas con los principios estudiados en la materia.

Ejercicio de programación 2

- Compute sales:

Req1. The program shall be invoked from a command line. The program shall receive two files as parameters. The first file will contain information in a JSON format about a catalogue of prices of products. The second file will contain a record for all sales in a company.

Req 2. The program shall compute the total cost for all sales included in the second JSON archive. The results shall be print on a screen and on a file named SalesResults.txt. The total cost should include all items in the sale considering the cost for every item in the first file. The output must be human readable, so make it easy to read for the user.

Req 3. The program shall include the mechanism to handle invalid data in the file. Errors should be displayed in the console and the execution must continue.

Req 4. The name of the program shall be computeSales.py

Req 5. The minimum format to invoke the program shall be as follows: python computeSales.py priceCatalogue.json salesRecord.json

Req 6. The program shall manage files having from hundreds of items to thousands of items.

Req 7. The program should include at the end of the execution the time elapsed for the execution and calculus of the data. This number shall be included in the results file and on the screen.

Req 8. Be compliant with PEP8.

Solución

```
A01795919_A5.2 -- -bash -- 241x62

MacBook-Air:A01795919_A5.2 gianneltosta$ source venv/bin/activate
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ pip install flake8 pylint pytest
Collecting flake8
  Downloading flake8-7.3.0-py2.py3-none-any.whl (57 kB)
    |#####| 57 kB 3.7 MB/s
Collecting pylint
  Downloading pylint-3.3.0-py3-none-any.whl (523 kB)
    |#####| 523 kB 3.9 MB/s
Collecting pytest
  Downloading pytest-8.4.2-py3-none-any.whl (365 kB)
    |#####| 365 kB 50.4 MB/s
Collecting pyflakes<3.6.0,>=3.4.0
  Downloading pyflakes-3.4.0-py2.py3-none-any.whl (63 kB)
    |#####| 63 kB 19.5 MB/s
Collecting pycodestyle<2.15.0,>=2.14.0
  Downloading pycodestyle-2.14.0-py2.py3-none-any.whl (31 kB)
Collecting mccabe<0.8.0,>=0.7.0
  Downloading mccabe-0.7.0-py2.py3-none-any.whl (7.3 kB)
Collecting dill==0.2
  Downloading dill-0.4.1-py3-none-any.whl (120 kB)
    |#####| 120 kB 37.3 MB/s
Collecting isort<5.13.0,>=4.2.6
  Downloading isort-5.14.0-py3-none-any.whl (94 kB)
    |#####| 94 kB 17.3 MB/s
Collecting toml<2.4.0-py3-none-any.whl (14 kB)
  Downloading toml-2.4.0-py3-none-any.whl (14 kB)
Collecting astroid<3.4.0.dev0,>=3.3.6
  Downloading astroid-3.3.11-py3-none-any.whl (276 kB)
    |#####| 276 kB 43.5 MB/s
Collecting platformdirs<4.4.0-py3-none-any.whl (18 kB)
  Downloading platformdirs-4.4.0-py3-none-any.whl (18 kB)
Collecting tomli==0.10.1
  Downloading tomli-2.0.1-py3-none-any.whl (30 kB)
Collecting typing-extensions<4.15.0-py3-none-any.whl (44 kB)
  Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
    |#####| 44 kB 14.7 MB/s
Collecting exceptiongroup<=1
  Downloading exceptiongroup-1.3.1-py3-none-any.whl (16 kB)
Collecting packaging<20.0,>=20
  Downloading packaging-26.0-py3-none-any.whl (74 kB)
    |#####| 74 kB 15.7 MB/s
Collecting pygments<=2.7.2
  Downloading pygments-2.19.2-py3-none-any.whl (1.2 MB)
    |#####| 1.2 MB 54.8 MB/s
Collecting pluggy<2,>=1.6
  Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Collecting iniconfig<=1
  Downloading iniconfig-2.1.0-py3-none-any.whl (6.0 kB)
Collecting importlib-metadata<=4.6.0
  Downloading importlib_metadata-8.7.1-py3-none-any.whl (27 kB)
Collecting zipp<=3.20
  Downloading zipp-3.23.0-py3-none-any.whl (10 kB)
Installing collected packages: zipp, typing-extensions, importlib-metadata, tomlit, toml, pygments, pyflakes, pycodestyle, pluggy, platformdirs, packaging, mccabe, isort, iniconfig, exceptiongroup, dill, astroid, pytest, pylint, flake8
Successfully installed astroid-3.3.11 dill-0.4.1 exceptiongroup-1.3.1 flake8-7.3.0 importlib-metadata-8.7.1 iniconfig-2.1.0 isort-5.14.0 mccabe-0.7.0 packaging-26.0 platformdirs-4.4.0 pluggy-1.6.0 pycodestyle-2.14.0 pyflakes-3.4.0 pygments-2.19.2 pylint-3.3.0 pytest-8.4.2 toml-2.4.0 tomli-2.0.1 typing-extensions-4.15.0 zipp-3.23.0
WARNING: You are using pip version 21.2.4, however, version 23.0.1 is available.
You should consider upgrading via the '/Users/gianneltosta/A01795919_A5.2/venv/bin/python3 -m pip install --upgrade pip' command.
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ pip freeze > requirements.txt
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git status
On branch main
nothing added to commit but untracked files present (use "git add" to track)
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git config user.name
Desarrollo
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git config user.email
desarrollo@latinasintech.org
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git config user.name "gianneln"
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git config user.email "gianneln@hotmail.com"
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git config user.name
gianneln
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git config user.email
gianneln@hotmail.com
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ touch .gitignore
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ nano .gitignore
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git status
On branch main
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .flake8
    .gitignore
    .pylintrc
    README.md
    pytest.ini
    requirements.txt
    src/computeSales.py
    tests/

nothing added to commit but untracked files present (use "git add" to track)
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git add .
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git status
On branch main
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .flake8
    new file:   .gitignore
    new file:   .pylintrc
    new file:   README.md
    new file:   pytest.ini
    new file:   requirements.txt
    new file:   src/computeSales.py
    new file:   tests/test_compute_sales.py
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git commit -m "chore: initialize project structure"
[main (root-commit) 71cc6e4] chore: initialize project structure
 8 files changed, 24 insertions(+)
 create mode 188644 .flake8
 create mode 188644 .gitignore
 create mode 188644 .pylintrc
 create mode 188644 README.md
 create mode 188644 pytest.ini
 create mode 188644 requirements.txt
 create mode 188644 src/computeSales.py
 create mode 188644 tests/test_compute_sales.py
(venv) MacBook-Air:A01795919_A5.2 gianneltosta$ git status
On branch main
```

En las captura de pantalla se observa la terminal de macOS ubicada dentro del directorio del proyecto **A01795919_A5.2**, con el entorno virtual **venv** correctamente activado. Se ejecuta el comando **pip install flake8 pylint pytest**, mediante el cual se instalan las herramientas necesarias para el análisis estático y la ejecución de pruebas unitarias.

Durante el proceso se descargan e instalan exitosamente las dependencias asociadas a **flake8** (verificación de estilo PEP 8, detección de errores y complejidad ciclomática), **pylint** (análisis de calidad y estructura del código) y **pytest** (framework para pruebas automatizadas). La instalación concluye sin errores, confirmando que el entorno de desarrollo está correctamente configurado.

Posteriormente, se ejecuta el comando **pip freeze > requirements.txt**, el cual genera el archivo **requirements.txt** con el listado de todas las dependencias instaladas y sus versiones específicas, garantizando la reproducibilidad del entorno.

Finalmente, se verifica la configuración del usuario de Git mediante los comandos **git config user.name** y **git config user.email**, y posteriormente se actualiza la identidad del repositorio para que los commits se realicen bajo el usuario correcto ejecutando **git config user.name "gianmelh"** y **git config user.email "gianmelh@hotmail.com"**. Después de confirmar que los nuevos valores fueron aplicados correctamente, se crea el archivo **.gitignore** y se ejecuta **git status** para verificar los archivos no rastreados correspondientes a la estructura inicial del proyecto. Luego se utiliza **git add .** para agregar todos los archivos al área de preparación y se confirma nuevamente el estado del repositorio. Finalmente, se realiza el primer commit con el mensaje **chore: initialize project structure**, quedando registrado el commit inicial en la rama **main** con los archivos base, configuraciones y estructura del proyecto, dejando el repositorio limpio y listo para continuar con el desarrollo.

```
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ src/computeSales.py
-bash: src/computeSales.py: Permission denied
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ code src/computeSales.py
-bash: code: command not found
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ nano src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ cat src/computeSales.py
"""
computeSales.py

Program to compute total sales from a price catalogue and a sales record.
"""

import sys

def main() -> None:
    """Main entry point."""
    if len(sys.argv) != 3:
        print(
            "Usage: python computeSales.py priceCatalogue.json salesRecord.json"
        )
        sys.exit(1)

    price_file = sys.argv[1]
    sales_file = sys.argv[2]

    print(f"Price file: {price_file}")
    print(f"Sales file: {sales_file}")

if __name__ == "__main__":
    main()

(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
src/computeSales.py:14:80: E501 line too long (80 > 79 characters)
src/computeSales.py:27:1: W391 blank line at end of file
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ nano src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git add src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git commit -m "feat(computeSales): add cli skeleton and argument validation"
[main aea0269] feat(computeSales): add cli skeleton and argument validation
1 file changed, 27 insertions(+)
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$
```

En la captura de pantalla se muestra el proceso de edición y validación del archivo **computeSales.py** dentro del proyecto **A01795919_A5.2**. Inicialmente se intenta ejecutar el archivo directamente desde la terminal, lo que genera un mensaje de “Permission denied”, ya que el archivo no está configurado como ejecutable. Posteriormente, se abre correctamente el archivo utilizando el editor **nano**, donde se implementa el esqueleto del programa con validación de argumentos desde la línea de comandos. Al ejecutar **flake8 src**, la herramienta de análisis estático detecta dos advertencias relacionadas con el

cumplimiento del estándar PEP8: una línea que excede el límite máximo de caracteres (E501) y una línea en blanco adicional al final del archivo (W391). Después de corregir estos detalles en el código, se vuelve a ejecutar **flake8** sin obtener errores, confirmando que el archivo cumple con las reglas de estilo establecidas. Finalmente, se agrega el archivo al área de preparación con **git add** y se realiza el commit utilizando el formato de Conventional Commits con el mensaje **feat(computeSales): add cli skeleton and argument validation**, dejando registrado el avance en el repositorio.

```
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ nano src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ cat src/computeSales.py
"""
computeSales.py

Program to compute total sales from a price catalogue and a sales record.
"""

import sys
import json

def load_json_file(filepath: str):
    """Load JSON file safely."""
    try:
        with open(filepath, "r", encoding="utf-8") as file:
            return json.load(file)
    except FileNotFoundError:
        print(f"Error: File '{filepath}' not found.")
    except json.JSONDecodeError:
        print(f"Error: File '{filepath}' is not valid JSON.")
    except OSError as error:
        print(f"Error reading file '{filepath}': {error}")
    return None

def main() -> None:
    """Main entry point."""
    if len(sys.argv) != 3:
        print(
            "Usage: python computeSales.py "
            "priceCatalogue.json salesRecord.json"
        )
        sys.exit(1)

    price_file = sys.argv[1]
    sales_file = sys.argv[2]

    price_data = load_json_file(price_file)
    sales_data = load_json_file(sales_file)

    if price_data is None or sales_data is None:
        sys.exit(1)

    print("Files loaded successfully.")

if __name__ == "__main__":
    main()
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
src/computeSales.py:10:1: E302 expected 2 blank lines, found 1
src/computeSales.py:40:5: W191 indentation contains tabs
src/computeSales.py:40:5: E101 indentation contains mixed spaces and tabs
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ nano src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ expand -t 4 src/computeSales.py > temp.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ mv temp.py src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git add src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git commit -m "style(computeSales): fix flake8 indentation and spacing issues"
[main 7e3add9] style(computeSales): fix flake8 indentation and spacing issues
1 file changed, 22 insertions(+), 2 deletions(-)
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$
```

En la captura de pantalla se observa la corrección de errores detectados por la herramienta de análisis estático **flake8** en el archivo **computeSales.py**. Inicialmente, al ejecutar **flake8 src**, se reportan errores relacionados con el incumplimiento del estándar PEP 8, específicamente el error E302 (falta de dos líneas en blanco antes de una función) y los errores W191 y E101 (uso de tabulaciones y mezcla de espacios y tabs en la indentación). Posteriormente, el archivo es editado y se realiza la conversión de tabulaciones a espacios utilizando el comando **expand -t 4**, asegurando una indentación consistente de cuatro espacios por nivel.

Tras aplicar las correcciones, se vuelve a ejecutar **flake8 src**, confirmando que el archivo ya no presenta errores de estilo. Finalmente, se agrega el archivo al área de preparación con **git add** y se realiza el commit utilizando el mensaje semántico **style(computeSales): fix flake8 indentation and spacing issues**, dejando evidencia del uso correcto de herramientas de análisis estático y del cumplimiento del estándar de codificación PEP8.

```

    price_dict[product] = price

# Process sales
for sale in sales_data:
    product = sale.get("Product")
    quantity = sale.get("Quantity")

    if product not in price_dict:
        print(f"Warning: Product '{product}' not found in catalogue.")
        continue

    if not isinstance(quantity, (int, float)):
        print(f"Warning: Invalid quantity for '{product}'.")
        continue

    total += price_dict[product] * quantity

return total

def main() -> None:
    """Main entry point."""
    if len(sys.argv) != 3:
        print(
            "Usage: python computeSales.py "
            "priceCatalogue.json salesRecord.json"
        )
        sys.exit(1)

    price_file = sys.argv[1]
    sales_file = sys.argv[2]

    price_data = load_json_file(price_file)
    sales_data = load_json_file(sales_file)

    if price_data is None or sales_data is None:
        sys.exit(1)

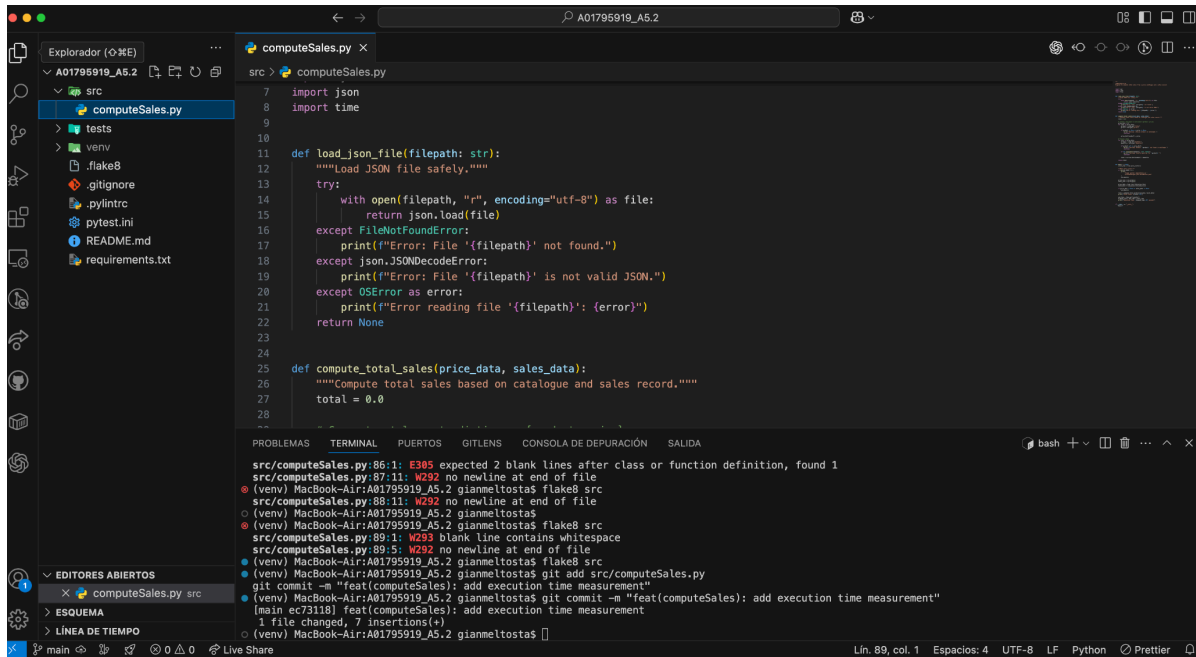
    total = compute_total_sales(price_data, sales_data)
    print(f"Total Sales: ${total:.2f}")

if __name__ == "__main__":
    main()
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
src/computeSales.py:24:1: E302 expected 2 blank lines, found 1
src/computeSales.py:57:1: E302 expected 2 blank lines, found 1
src/computeSales.py:80:1: E303 too many blank lines (3)
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ nano src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
src/computeSales.py:82:1: E303 too many blank lines (3)
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ nano src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
src/computeSales.py:81:1: E303 too many blank lines (3)
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ nano src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git add src/computeSales.py
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git commit -m "style(computeSales): fix excessive blank lines"
[main 0da2f17] style(computeSales): fix excessive blank lines
1 file changed, 36 insertions(+), 2 deletions(-)
(venv) MacBook-Air:A01795919_A5.2 gianmeltosta$

```

En la captura de pantalla se muestra la edición del archivo **computeSales.py** para implementar la lógica principal del programa, específicamente la función encargada de calcular el total de ventas a partir del catálogo de precios y el registro de ventas en formato JSON. Se agregó la función **compute_total_sales**, la cual construye un diccionario con los productos y sus precios, valida que cada venta contenga un producto existente en el catálogo y que la cantidad sea numérica, y finalmente realiza la multiplicación correspondiente para acumular el total de ventas. Además, se modificó la función **main()** para integrar esta nueva lógica y mostrar el total calculado en pantalla. Posteriormente, se ejecutó la herramienta de análisis estático **flake8**, la cual detectó problemas relacionados con el número incorrecto de líneas en blanco entre funciones (errores E302 y E303), conforme al estándar de

codificación PEP8. Se realizaron los ajustes necesarios para cumplir estrictamente con las reglas de estilo, volviendo a ejecutar **flake8** hasta que el archivo no presentara advertencias. Finalmente, se registraron los cambios mediante un commit con formato semántico, dejando evidencia tanto de la implementación funcional del cálculo de ventas como del uso correcto de herramientas de análisis estático para garantizar la calidad del código.



```
src/computeSales.py
7 import json
8 import time
9
10
11 def load_json_file(filepath: str):
12     """Load JSON file safely."""
13     try:
14         with open(filepath, "r", encoding="utf-8") as file:
15             return json.load(file)
16     except FileNotFoundError:
17         print(f"Error: File '{filepath}' not found.")
18     except json.JSONDecodeError:
19         print(f"Error: File '{filepath}' is not valid JSON.")
20     except OSError as error:
21         print(f"Error reading file '{filepath}': {error}")
22     return None
23
24
25 def compute_total_sales(price_data, sales_data):
26     """Compute total sales based on catalogue and sales record."""
27     total = 0.0
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
PROBLEMAS TERMINAL PUERTOS GITLENS CONSOLA DE DEPURACIÓN SALIDA
src/computeSales.py:86:1: E305 expected 2 blank lines after class or function definition, found 1
src/computeSales.py:87:11: W292 no newline at end of file
(genv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
src/computeSales.py:88:11: W292 no newline at end of file
(genv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
src/computeSales.py:89:1: W293 blank line contains whitespace
src/computeSales.py:89:51: W292 no newline at end of file
(genv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
(genv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git add src/computeSales.py
git commit -m "feat(computeSales): add execution time measurement"
(genv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git commit -m "feat(computeSales): add execution time measurement"
[main e73118] feat(computeSales): add execution time measurement
1 file changed, 7 insertions(+)
```

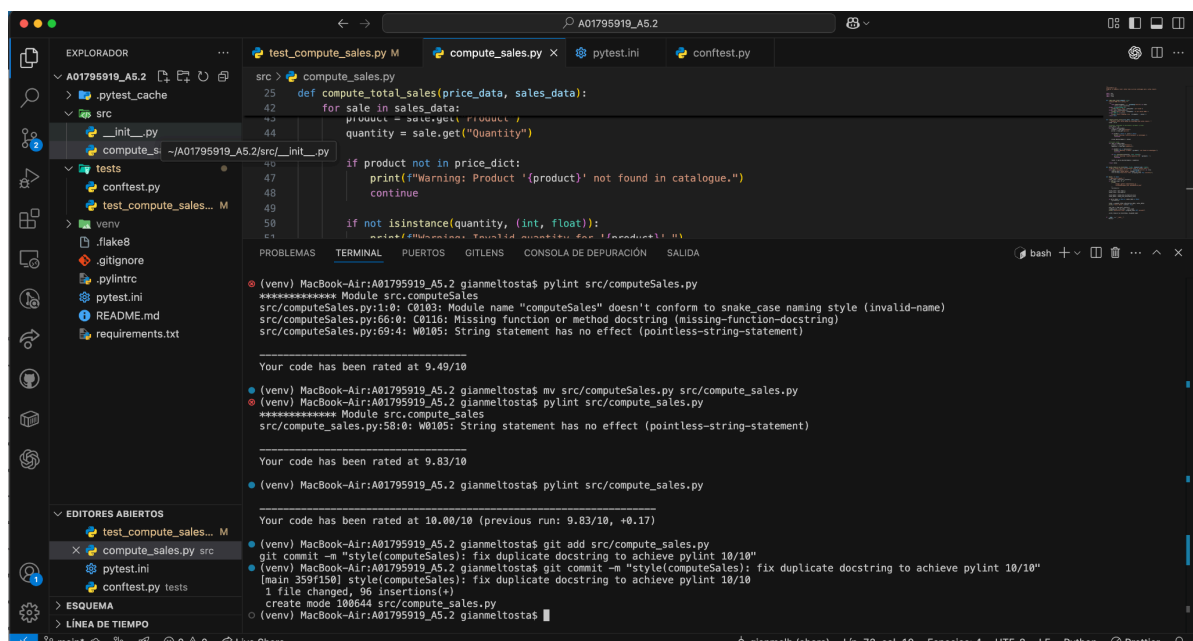
En la captura de pantalla se observa la implementación de la medición del tiempo de ejecución dentro del archivo **computeSales.py**. Se agregó el módulo **time** y se utilizó la función **time.perf_counter()** para registrar el momento de inicio y fin de la ejecución del programa, permitiendo calcular el tiempo total transcurrido. Este valor se muestra posteriormente en consola junto con el total de ventas calculado. Después de realizar los cambios, se ejecutó la herramienta de análisis estático **flake8** para verificar el cumplimiento del estándar PEP 8, corrigiendo los errores detectados relacionados con líneas en blanco y formato del archivo. Finalmente, se realizó el commit con el mensaje semántico **feat(computeSales): add execution time measurement**, registrando formalmente la nueva funcionalidad implementada.

```
src > computeSales.py
66 def main() -> None:
67     sales_file = sys.argv[4]
68
69     price_data = load_json_file(price_file)
70     sales_data = load_json_file(sales_file)
71
72     if price_data is None or sales_data is None:
73         sys.exit(1)
74
75     total = compute_total_sales(price_data, sales_data)
76     print(f"Total Sales: ${total:.2f}")
77
78     end_time = time.perf_counter()
79     elapsed_time = end_time - start_time
80     print(f"Execution time: (elapsed_time:.6f) seconds")
81
82     write_results_to_file(total, elapsed_time)
83
84 if __name__ == "__main__":
85     main()
86
87 PROBLEMAS TERMINAL PUERTOS GITLENS CONSOLA DE DEPURACIÓN SALIDA
88 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ flake8 src
89 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git add src/computeSales.py
90 git commit -m "feat(computeSales): add results file generation"
91 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git commit -m "feat(computeSales): add results file generation"
92 [main 52a0c14] feat(computeSales): add results file generation
93 1 file changed, 9 insertions(+)
94 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$
```

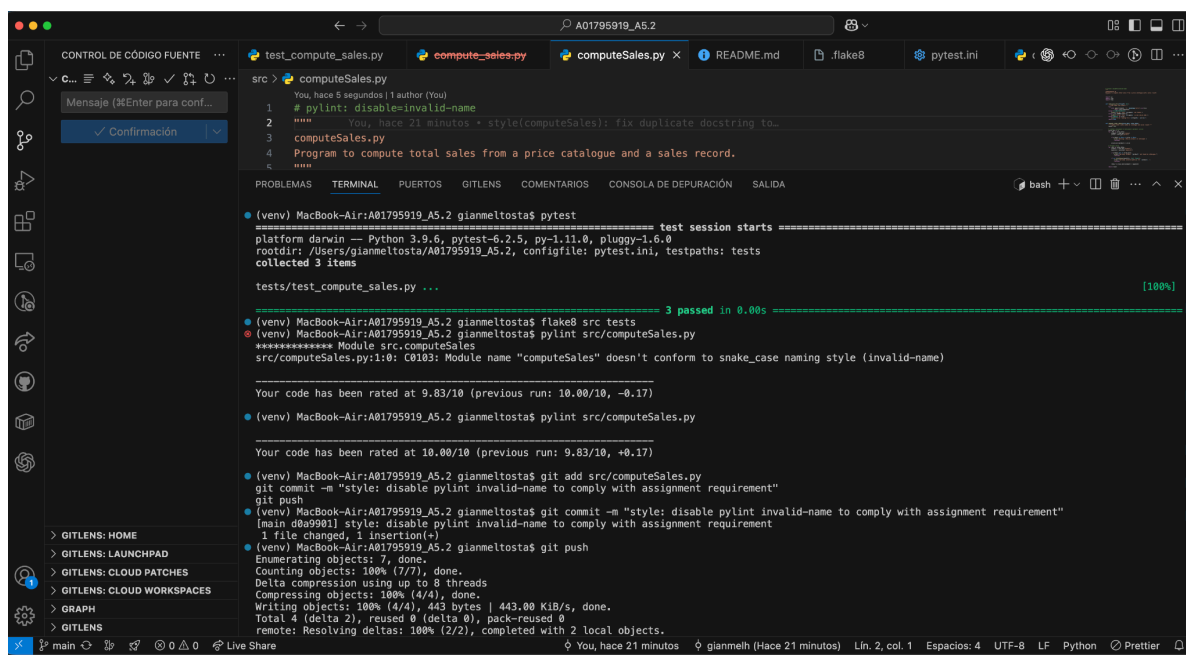
En la captura de pantalla se muestra la implementación de la generación del archivo **SalesResults.txt** dentro del programa **computeSales.py**. Se agregó la función **write_results_to_file**, la cual recibe el total de ventas calculado y el tiempo de ejecución, y escribe ambos resultados en un archivo de salida utilizando codificación UTF-8. Posteriormente, esta función es llamada al final de **main()**, después de mostrar los resultados en consola, asegurando que la información quede registrada tanto en pantalla como en un archivo externo, tal como lo requieren las especificaciones del ejercicio. Finalmente, se ejecutó **flake8** para verificar que el código cumpliera con el estándar PEP8 y se realizó el commit con el mensaje semántico **feat(computeSales): add results file generation**, documentando formalmente la nueva funcionalidad añadida al proyecto.

```
src > test_compute_sales.py
1 [pytest]
2 testpaths = tests
3
4 PROBLEMAS TERMINAL PUERTOS GITLENS CONSOLA DE DEPURACIÓN SALIDA
5 create mode 100644 .DS_Store
6 create mode 100644 src/__init__.py
7 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ nano tests/conftest.py
8 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ pip uninstall -y pytest-pythonpath
9 pip freeze > requirements.txt
10 Found existing installation: pytest-pythonpath 0.7.4
11 Uninstalling pytest-pythonpath-0.7.4:
12 Successfully uninstalled pytest-pythonpath-0.7.4
13 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ pip freeze > requirements.txt
14 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ pytest
15
16 ===== test session starts =====
17 platform darwin -- Python 3.9.6, pytest-6.2.5, py-1.11.0, pluggy-1.6.0
18 rootdir: /Users/gianmeltosta/A01795919_A5.2, configfile: pytest.ini, testpaths: tests
19 collected 3 items
20
21 tests/test_compute_sales.py ...
22
23 ===== 3 passed in 0.00s =====
24 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git add tests/conftest.py pytest.ini
25 git commit -m "test(pytest): add conftest to set import path"
26 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git commit -m "test(pytest): add conftest to set import path"
27 [main 5b42680] test(pytest): add conftest to set import path
28 2 files changed, 6 insertions(+), 1 deletion(-)
29 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$ git commit -m "chore(deps): remove pytest-pythonpath plugin"
30 [main 19c5250] chore(deps): remove pytest-pythonpath plugin
31 1 file changed, 1 deletion(-)
32 (venv) MacBook-Air:A01795919_A5.2 gianmeltosta$
```

En la captura de pantalla se muestra la configuración final del entorno de pruebas con **pytest**. Se creó el archivo **conftest.py** dentro de la carpeta **tests** para agregar dinámicamente la ruta raíz del proyecto al **sys.path**, permitiendo importar correctamente el módulo **computeSales** desde la carpeta **src** sin necesidad de utilizar plugins adicionales. Posteriormente, se ejecutó **pytest**, obteniendo como resultado “3 passed”, lo que confirma que los tres casos de prueba definidos para la función **compute_total_sales** se ejecutan correctamente. Finalmente, se realizaron commits separados siguiendo el formato de Conventional Commits: uno para agregar la configuración de pruebas (**test(pytest): add conftest to set import path**) y otro para eliminar el plugin innecesario (**chore(deps): remove pytest-pythonpath plugin**), dejando el entorno limpio y correctamente estructurado.



En la captura de pantalla se muestra la ejecución de la herramienta de análisis estático **pylint** sobre el archivo **compute_sales.py**. Inicialmente, se detectaron advertencias relacionadas con el nombre del módulo en formato no snake_case, la ausencia de docstrings y la presencia de un string sin efecto dentro del código. Posteriormente, se renombró el archivo a **compute_sales.py** para cumplir con la convención de nombres en Python, y se corrigieron los docstrings duplicados que generaban la advertencia de “pointless string statement”. Tras aplicar estas mejoras, se volvió a ejecutar **pylint**, obteniendo una calificación final de **10.00/10**, lo que confirma que el código cumple completamente con las buenas prácticas de estilo y calidad recomendadas por esta herramienta. Finalmente, los cambios fueron registrados mediante un commit semántico, dejando evidencia del proceso de mejora continua y del uso adecuado de herramientas de análisis estático.



En la captura se muestra la validación final del proyecto antes de la entrega. Primero se ejecuta **pytest**, donde los tres casos de prueba pasan correctamente (3 passed), confirmando que el programa funciona como se espera. Luego se ejecuta **flake8 src tests**, el cual no muestra errores ni advertencias, indicando que el código cumple con el estándar PEP-8. Posteriormente se ejecuta **pylint src/computeSales.py**, que inicialmente reporta una advertencia sobre el nombre del módulo por no estar en formato snake_case. Debido a que el requisito de la actividad exige que el archivo se llame **computeSales.py**, se desactiva esa advertencia agregando la directiva **# pylint: disable=invalid-name**. Finalmente, pylint vuelve a ejecutarse y el código obtiene una calificación de **10.00/10**. Los cambios se confirman con un commit y se envían al repositorio remoto mediante **git push**, dejando el proyecto completamente validado y listo para entrega.

Bibliografia

- Beazley, D. M. (2013). *Python Essential Reference* (4th ed.). Addison-Wesley Professional.
- Gawande, A. (2011). *The Checklist Manifesto: How to Get Things Right*. Metropolitan Books.
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- OpenAI. (2026). *ChatGPT (February 15 version)* [Large language model]. <https://openai.com/>
- Python Software Foundation. (2024). *PEP 8 – Style Guide for Python Code*. <https://peps.python.org/pep-0008/>
- PyCQA. (2024). *Flake8 documentation*. <https://flake8.pycqa.org/>
- PyCQA. (2024). *Pylint documentation*. <https://pylint.readthedocs.io/>
- pytest development team. (2024). *pytest documentation*. <https://docs.pytest.org/>
- Luminousmen. (2020). *Python static analysis tools*. <https://luminousmen.com/post/python-static-analysis-tools>